a single web page. Your first view was the Credits page, showing the website's authors. This view used plain text; however, instead of returning plain text, most of your views will return HTML.

The Django Template Engine is a collection of tools that allow you to compose and control text in a way similar to how inheritance works in object-oriented programming. A header or footer can be declared in a separate file and then included in each page that is rendered. This isolates the text, and any changes automatically get propagated throughout your site.

On a web page with a navigation menu, you might want to highlight the current page. Instead of different header blocks for each page, Django includes conditional rendering. It supports the equivalent of Python `if` and `else` statements, allowing you to change the rendering of the navigation bar based on Python objects passed as arguments to the template engine.

> **NOTE** The template engine is not HTML specific—it operates on any text. The vast majority of the time, you will be using it to manage HTML, but you can also templatize email messages, personalize prefilled text in forms, or perform any other text manipulation you come up with.

The interface to the template engine is composed of two key objects, both found in the `django.template` module: `Template` and `Context`. The `Template` object represents a piece of text written in the template language, while the `Context` object provides data to the template engine for rendering the result.

3.2 Context objects and the parts of a Template

Using the Django Template Engine breaks down into three steps:

1. Create or load a template.
2. Define the context.
3. Render a template using the context.

When you create or load a template, you get a `django.template.Template` object as a result. The contents of this object are actually a compiled version of the text you want in the template. To get the output from the template, you need to render it. Rendering requires a `django.template.Context` object that contains any data used in the template. For example, if you are using a template for a navigation menu and want to highlight the current page, the context might contain a variable indicating the name of the current page. Inside the template, a conditional block checks the value of the current page variable and changes the rendering of the menu as needed. Pieces of templates can be composed together, so a template is often part of a page—it doesn't have to be the whole thing.

`Context` objects get built using dictionaries. Anything you can put inside a Python dictionary can be used as context. If the template references something not found